

From protocol code to checkable evidence

One reusable Rust-to-Lean pipeline.

Readable claims. Explicit assumptions. Replayable proofs.

3

working examples

14

example claims checked

12

regression tests passed

A general engine, demonstrated beyond Paxos

Adding a protocol now means adding its specification and proof adapter—not changing the driver.

Reusable engine

Rust library, source file or ZIP

One extraction and checking path

Readable contracts

Commented TOML or JSON

Preview meaning and assumptions

Auditable results

Exact propositions and logs

Proof replay with axiom checks

Delivered: code, specifications, examples, documentation and an agent authoring skill.

A small spec, with the assumption in plain sight

Descriptions explain intent. Structured fields determine the exact Lean proposition.

```
[system]
module = "Delivery.module"
assumptions = ["Delivery.Fair"]

[[claims]]
name = "eventual_delivery"
kind = "eventually"
predicate = "Delivery.Done"
proof = "Delivery.eventually_done"
```

1 Preview

Read the claim and its premises.

2 Verify

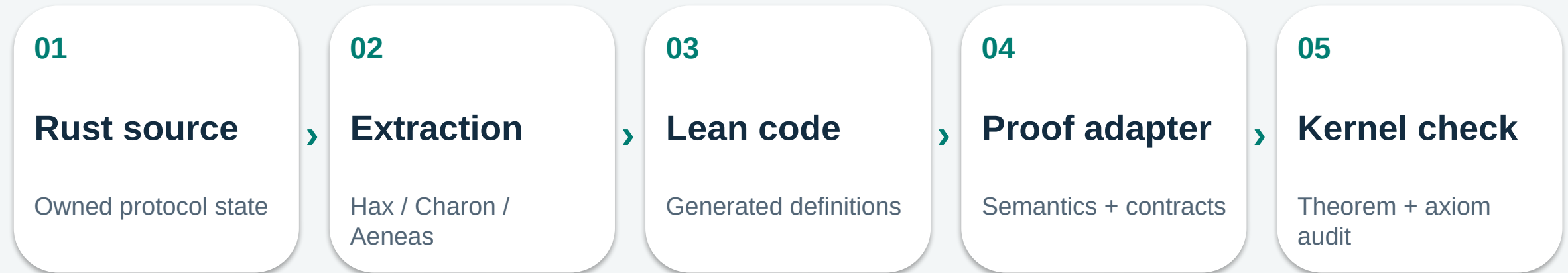
The crate finds its spec automatically.

3 Inspect & replay

Review the theorem, logs and evidence.

The proof is connected to the submitted Rust

A model theorem alone is not an implementation proof.



Saved evidence: source snapshot + specification + generated Lean + proofs + logs + hashes.

Three examples exercise different kinds of proof

All use the same driver. The example claims are checked without a finite execution-length bound.

Example	Checked claims	What this demonstrates
Paxos	4 / 4	Agreement, local progress, eventual choice, refinement
Delivery	5 / 5	All temporal forms; why fairness is needed
Pedersen	5 / 5	Correctness, hiding, binding reduction and bound

“Proved” applies to each precise proposition and its stated premises.

Paxos: agreement and conditional progress

Verified scope: the three-acceptor, single-decree Rust protocol core.

Safety

Successful learner certificates agree.
No fairness premise weakens this invariant.

Conditional liveness

A stable, fair quorum eventually chooses.
Enabled local computations also succeed.

Outside the proof: sockets, recovery, leader election and eventual client receipt.

Pedersen: perfect hiding and a binding reduction

The security theorems are linked to extracted generic Rust commitment and verification functions.

$$C = m \cdot G + r \cdot H$$

Hiding

Any two messages give identical commitment distributions under uniform blinding.

Binding

Two accepted, distinct openings yield $\log_G(H)$; a hardness bound transfers to forgery.

Scope: algebraic scheme + backend contracts; no curve, RNG, constant-time or asymptotic proof.

Proof failures are tested, too

12 regression tests passed in 450.6 seconds on this workstation.

Check	Observed result
Incorrect Paxos highest-ballot selector	Not proved
Paxos proposer always rejects	Unknown; liveness proof breaks
Pedersen blinding replaced with message	Unknown; refinement proof breaks
Delivery without fairness	Refuted, with a checked negation
A proof containing sorry	Rejected by axiom audit

Rust unit tests passed; shared specification semantics compiled on Lean 4.31 and 4.32.

A failed proof is not a counterexample

The engine reports evidence it actually established.

PROVED

Lean checked the proposition and its transitive axioms.

REFUTED

Lean checked the negation of that exact proposition.

UNKNOWN

Neither proof was established, or a proof/build timed out.

Extraction failures and input/tooling errors are reported separately.

A clear division of work for people and agents

The interface removes repetitive claim syntax while keeping proof obligations visible.

Protocol author

Write Rust and observable predicates.

Choose the property kind.

Proof author / agent

State environment assumptions.

Supply the adapter and lemmas.

Engine

Extract, elaborate, check and audit.

Save evidence for replay.

General Lean claims cover richer mathematics; automatic proof search remains deliberately limited.

Next: connect the verified core to a real runtime

Recommended priorities for turning the verified core into a deployed system.

01 Connect one real runtime

Prove the transport, ownership and recovery interface matches the model.

02 Instantiate the cryptographic backend

Discharge group-operation contracts; verify parameter handling and randomness.

03 Operationalize replay in CI

Pin toolchains, cache dependencies and retain proof evidence per revision.

Start with the delivery example

Preview → **verify** → **replay**

```
python3 formal/rust_verify.py --explain-spec \  
  formal/rust/delivery/spec.toml  
python3 formal/rust_verify.py formal/rust/delivery  
python3 formal/rust_verify.py --recheck <evidence-directory>
```

Read: formal/rust/SPECIFICATIONS.md

Agent guide: formal/skills/write-verification-spec/SKILL.md